

Systems Analysis and Design

Program Design

Alan Dennis, Barbara Haley Wixom, and Roberta Roth

Chapter 10 Outline

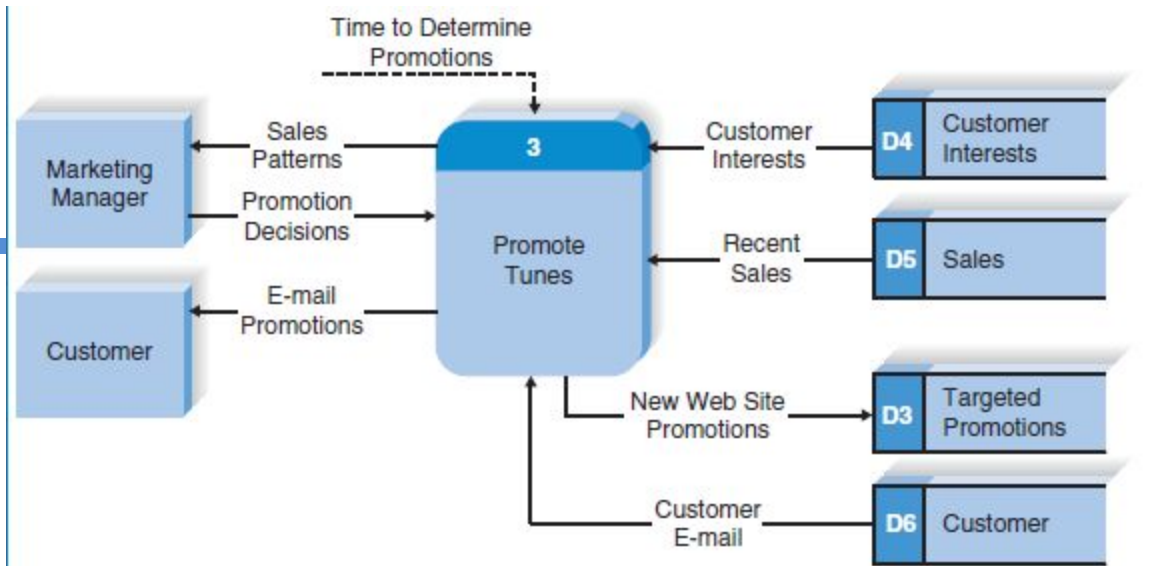
- Moving from logical to physical process models.
- Designing programs.
- Structure chart.
- Program specification.

INTRODUCTION

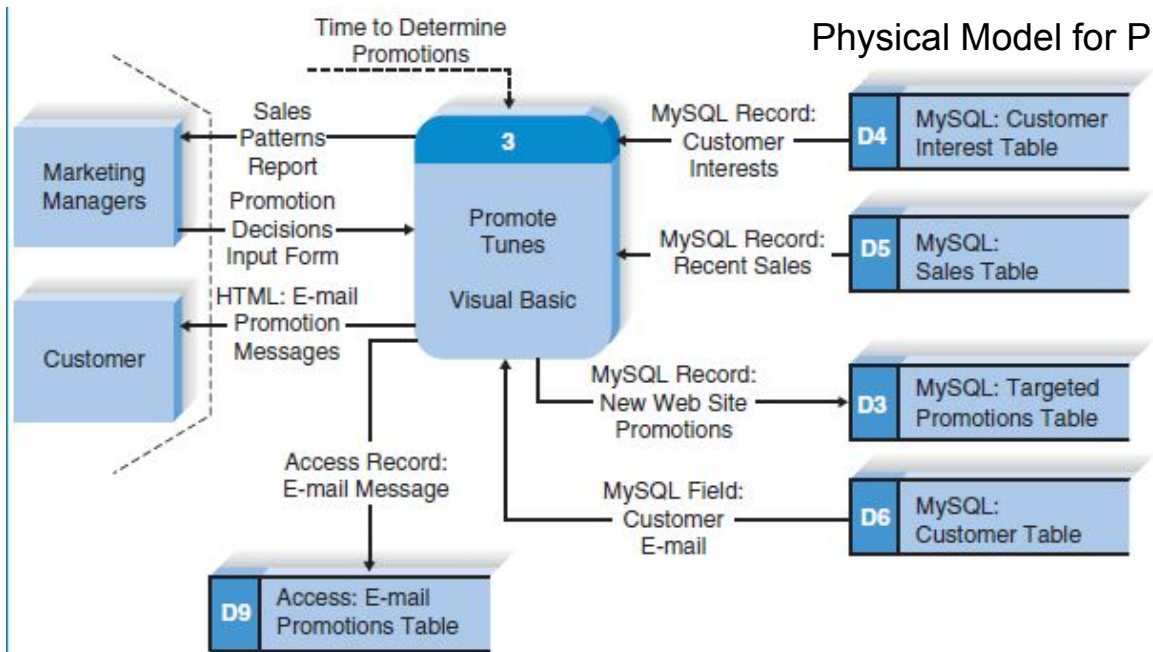
- **Program design** – Analysts determine what programs will be written and create instructions for the programmers.
- Various implementation decisions are made about the new system (e.g., what programming language(s) will be used.)
- The DFDs created during analysis are modified to show these implementation decisions, resulting in a set of *physical data flow diagrams*.
- The analysts determine how the processes are organized, using a *structure chart* to depict the decisions.
- Detailed instructions called *program specifications* are developed.

Steps to Create the Physical Data Flow Diagram

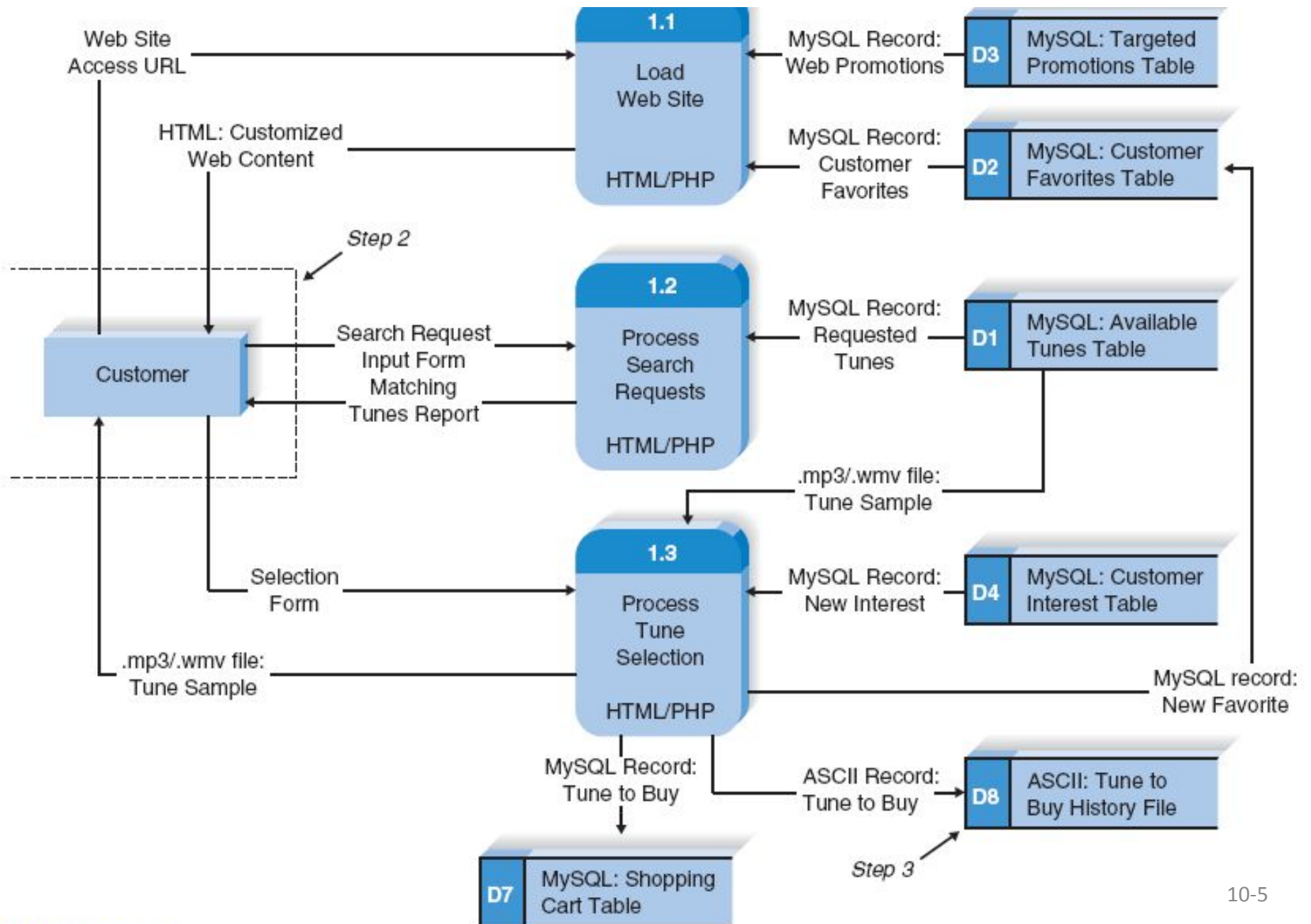
Step	Explanation
Add implementation references.	Using the existing logical DFD, add the way in which the data stores, data flows, and processes will be implemented to each component.
Draw a human-machine boundary.	Draw a line to separate the automated parts of the system from the manual parts.
Add system-related data stores, data flows, and processes.	Add system-related data stores, data flows, and processes to the model (components that have little to do with the business process).
Update the data elements in the data flows.	Update the data flows to include system-related data elements.
Update the metadata in the CASE repository.	Update the metadata in the CASE repository to include physical characteristics.
CASE = computer-aided software engineering; DFD = data flow diagram.	



Logical Model of the Promote Tunes Process



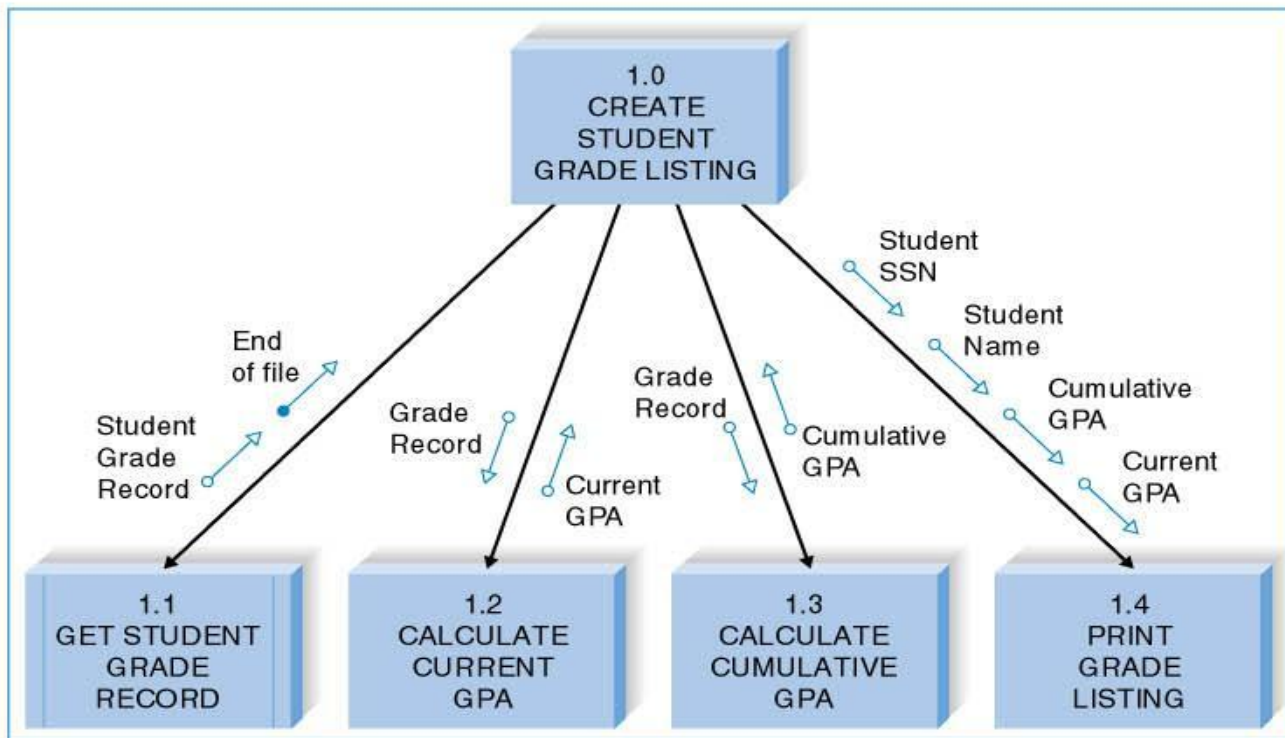
Physical Model for Promote Tunes Process



STRUCTURE CHART

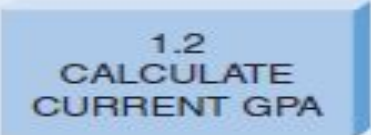
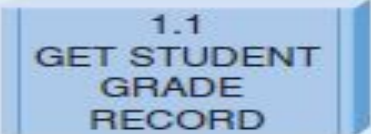
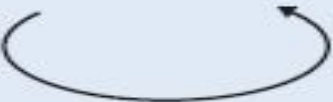
- The *structure chart* is an Important program design technique that help the analyst design the program.
- It shows all components of code in a hierarchical format that implies
 - *Sequence (in what order components are invoked),*
 - *Selection (under what condition a module is invoked),*
 - *Iteration (how often a component is repeated)*

An academic system needs a program that will print a listing of students along with their grade point averages (GPAs), both for the current semester and overall. First, the program must retrieve the student grade records; then it must calculate the current and cumulative GPAs; finally, the grade list can be printed.

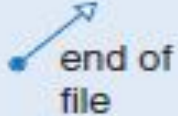


-
- a programmer can tell that there are four main code modules involved in creating a student grade listing: getting the student grade records, calculating current GPA, calculating cumulative GPA, and printing the listing. Also, there are various pieces of information that are either required by each module or created by it (e.g., the grade record, the cumulative GPA).

Syntax of Structure Chart

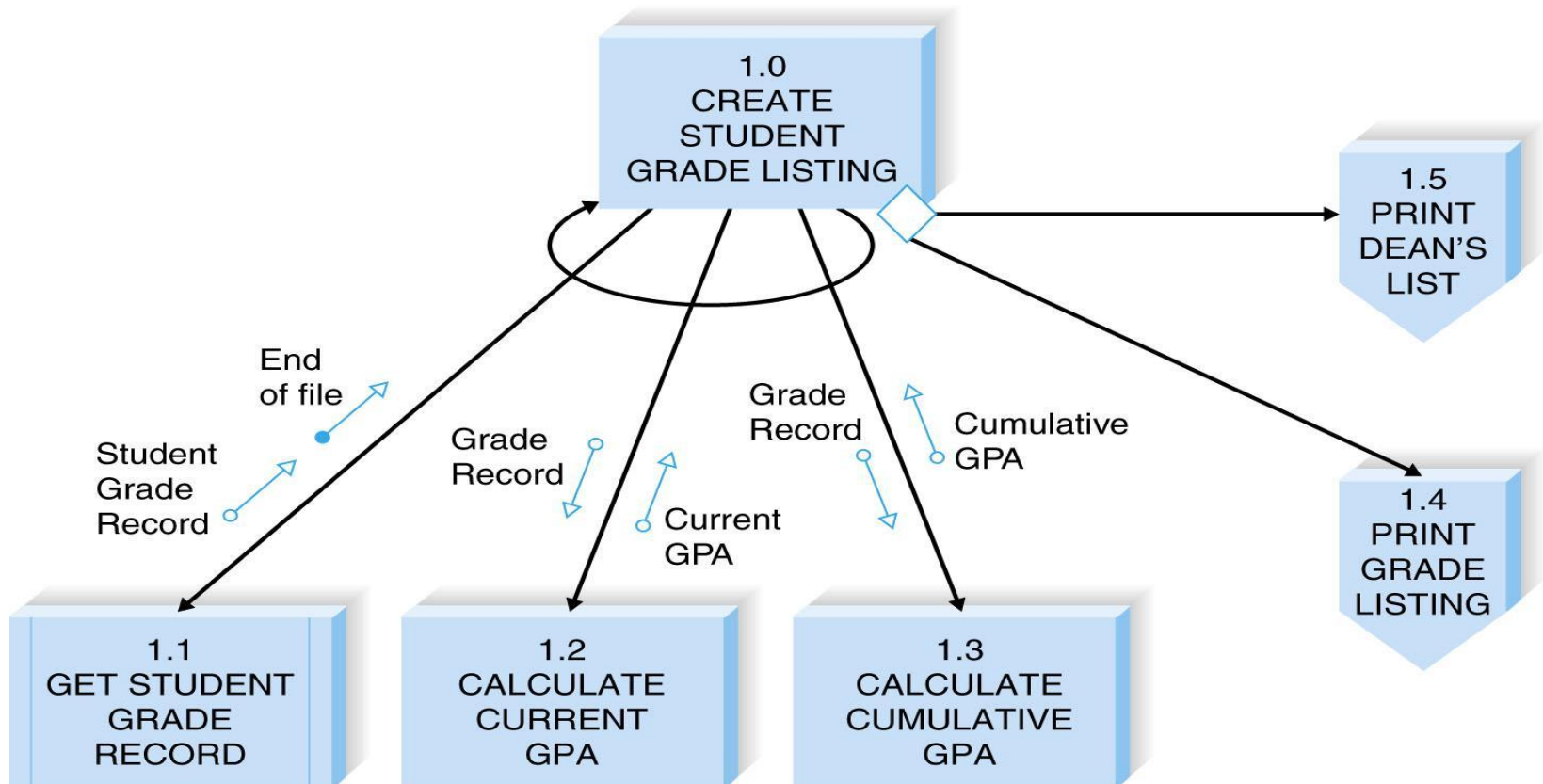
Structure Chart Element	Purpose	Symbol
Every <i>module</i>: • Has a number. • Has a name. • Is a control module if it calls other modules below it. • Is a subordinate module if it is controlled by a module at a higher level.	Denotes a logical piece of the program	
Every <i>library module</i> has: • A number. • A name. • Multiple instances within a diagram.	Denotes a logical piece of the program that is repeated within the structure chart	
A <i>loop</i>: • Is drawn with a curved arrow. • Is placed around lines of one or more modules that are repeated.	Communicates that a module(s) is repeated	
A <i>conditional line</i>: • Is drawn with a diamond. • Includes modules that are invoked on the basis of some condition.	Communicates that subordinate modules are invoked by the control module based on some condition	
A <i>data couple</i>: • Contains an arrow. • Contains an empty circle. • Names the type of data that are being passed. • Can be passed up or down. • Has a direction that is denoted by the arrow.	Communicates that data are being passed from one module to another	

(cont'd)

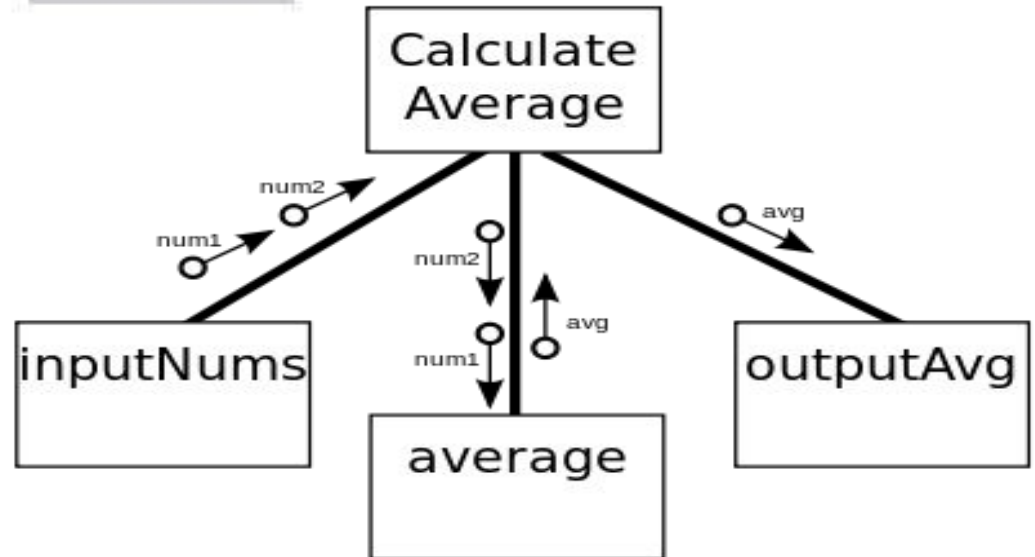
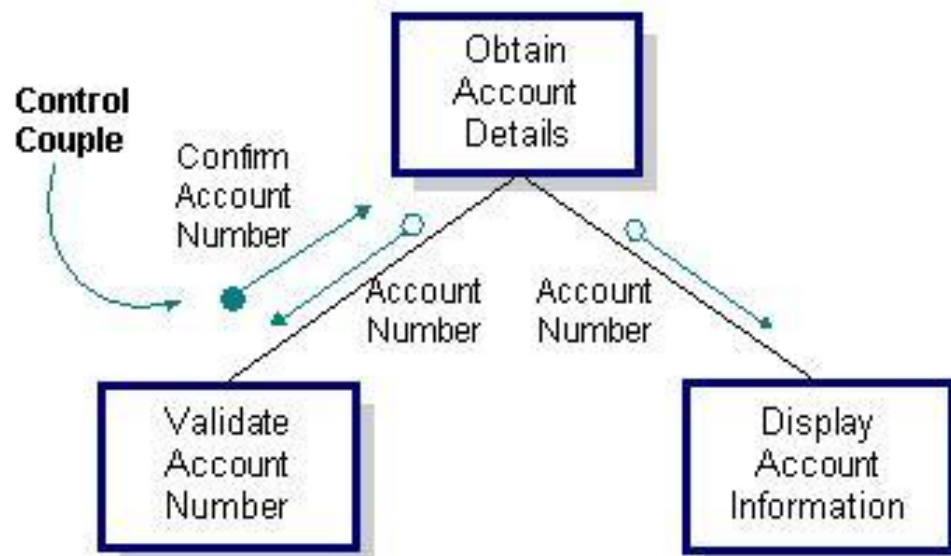
<i>A control couple:</i> • Contains an arrow. • Contains a filled-in circle. • Names the message or flag that is being passed. • Should be passed up, not down. • Has a direction that is denoted by the arrow.	Communicates that a message or a system flag is being passed from one module to another	
<i>An off-page connector:</i> • Is denoted by the hexagon. • Has a title. • Is used when the diagram is too large to fit everything on the same page.	Identifies when parts of the diagram are continued on another page of the structure chart	
<i>An on-page connector:</i> • Is denoted by the circle. • Has a title. • Is used when the diagram is too large to fit everything in the same spot on a page.	Identifies when parts of the diagram are continued somewhere else on the same page of the structure chart	

(cont'd)

Revised structure chart example



CONTROL COUPLE ON A STRUCTURE CHART



Design Guidelines

- High quality structure charts result in programs that are modular, reusable, and easy to implement.
- Measures of good design include
 - cohesion,
 - appropriate levels of fan-in and fan-out.

Build Modules with High Cohesion

- **Cohesion** refers to how well the lines of code within each module relate to each other.
- Cohesive modules are easy to understand and build because their code performs one function effectively.
 - High cohesion = easier to maintain, understand, and reuse.
 - Low cohesion = hard to manage and test

Types of Cohesion Overview

- Coincidental Cohesion
- Logical Cohesion
- Temporal Cohesion
- Procedural Cohesion
- Communicational Cohesion
- Sequential Cohesion
- Functional Cohesion

(Ordered from worst to best)

Coincidental Cohesion

- Definition: Random tasks grouped together.
- Example:

```
def random_tasks() :  
    open_file()  
    send_email()  
    draw_circle()
```

Logical Cohesion

- Tasks grouped by category but selected via flags.
- Example:

```
def handle_input(input_type, data):  
    if input_type == 'keyboard':  
        process_keyboard(data)  
    elif input_type == 'mouse':  
        process_mouse(data)
```

Temporal Cohesion

- Tasks executed at the same time.
- Example:

```
def initialize_system():  
    load_config()  
    connect_database()  
    setup_logging()
```

Procedural Cohesion

- Tasks done in a sequence.
- Example:

```
def process_file():  
    open_file()  
    read_file()  
    close_file()
```

Communicational Cohesion

- Tasks operate on the same data.
- Example:

```
def generate_report(data):  
    summary = calculate_summary(data)  
    chart = create_chart(data)  
    print_report(summary, chart)
```

Sequential Cohesion

- Output of one step is input to the next.
- Example:

```
def process_jpg(jpg):  
    loaded_jpg = load_jpg(jpg)  
    resized_jpg = resize_jpg(loaded_jpg)  
    upload_jpg (resized_jpg)
```

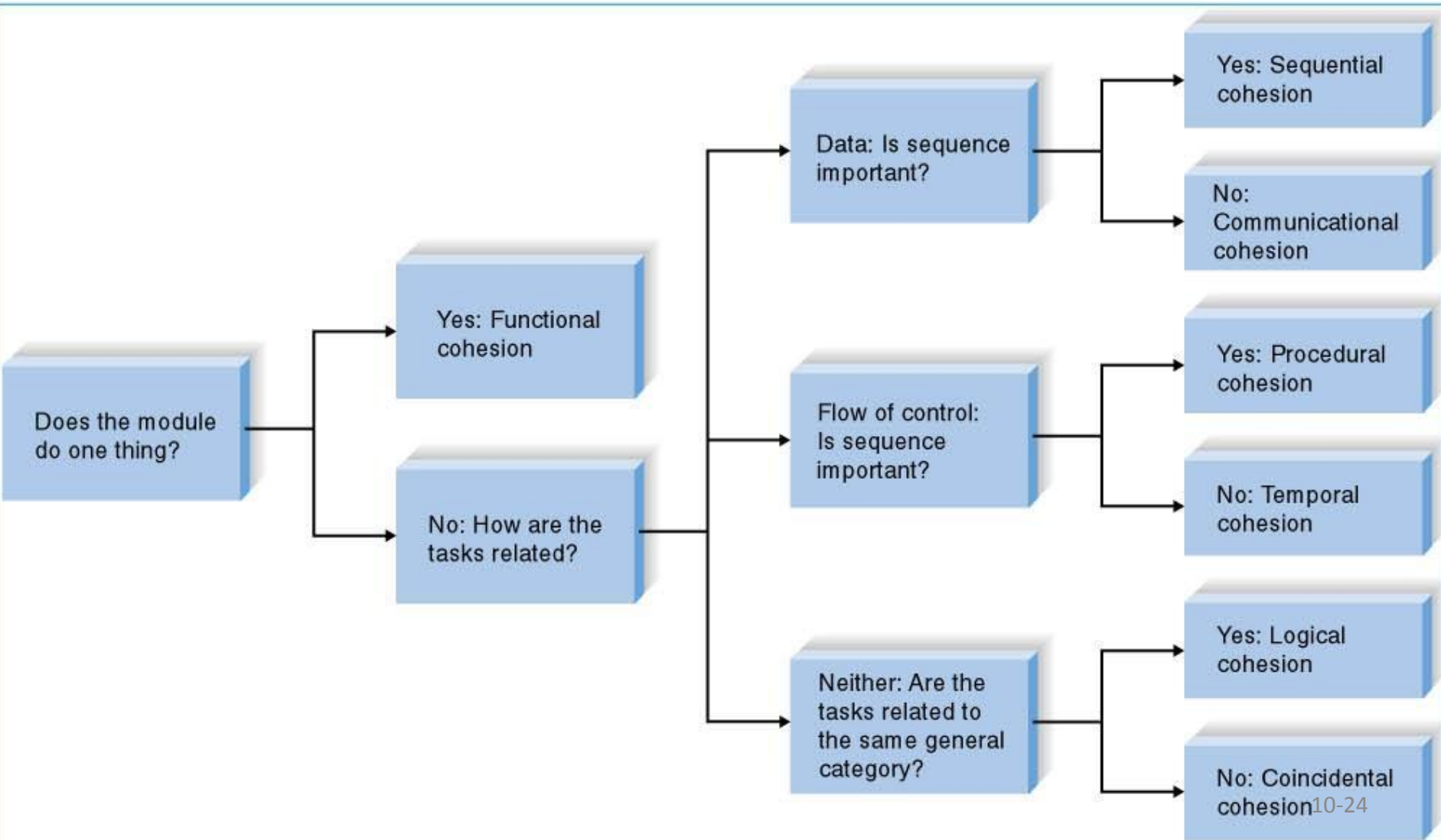
Functional Cohesion (Best)

- Every part contributes to one task.
- Example:

```
def calculate_average(numbers):  
    total = sum(numbers)  
    return total / len(numbers)
```

Bad

Cohesion Decision Tree



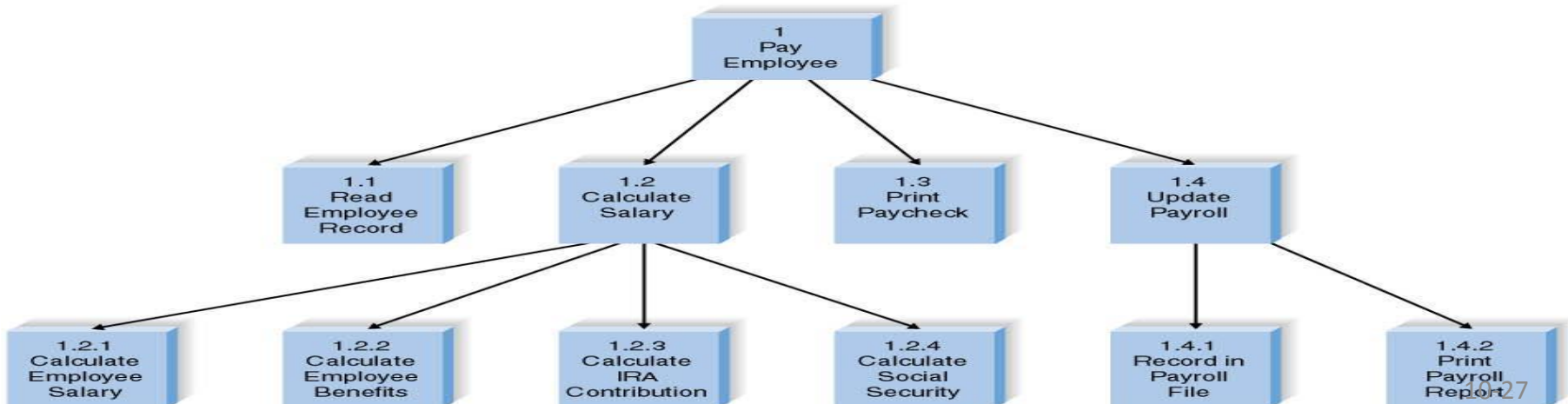
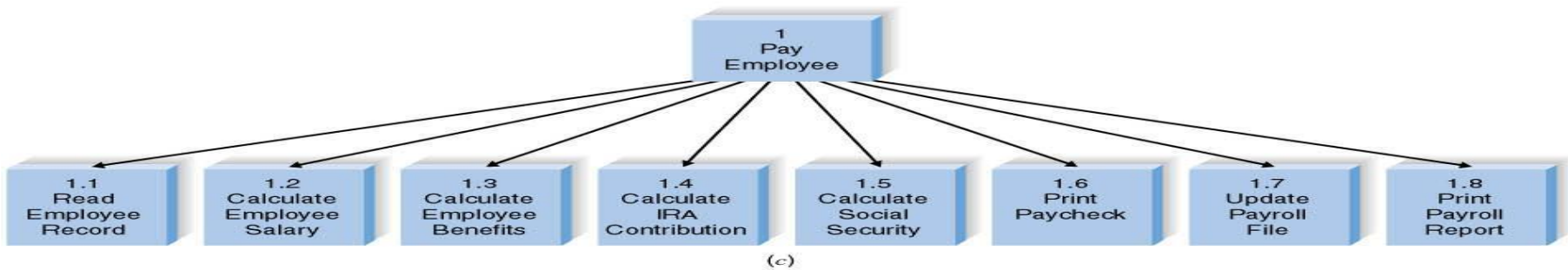
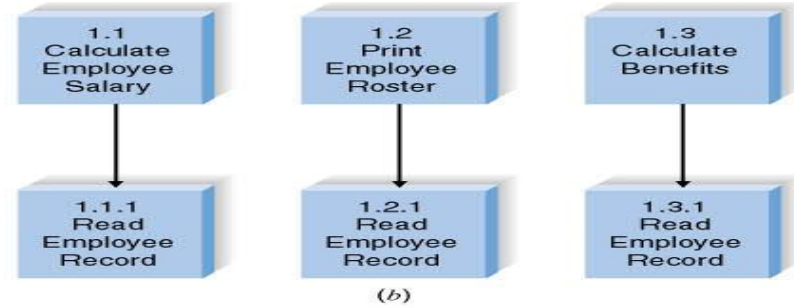
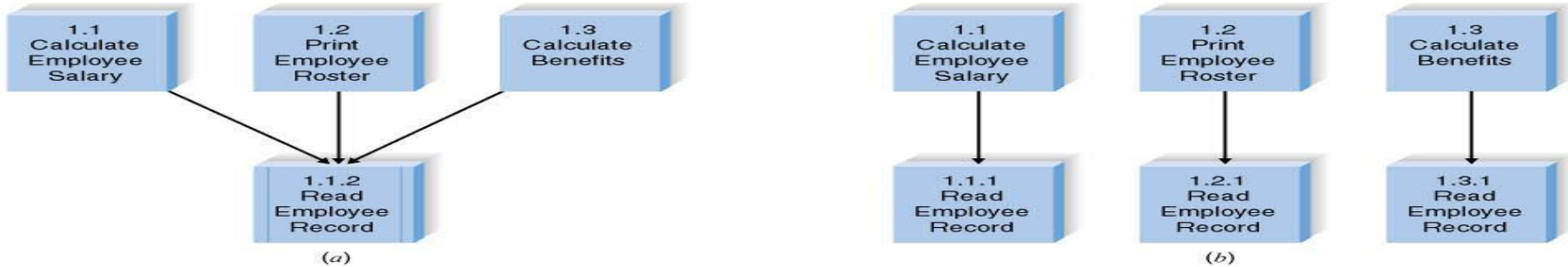
Fan-In

- *Fan-in* describes the number of control modules that communicate with a subordinate.
- A module with high fan-in has many different control modules that call it. This is a good situation because high fan-in indicates that a module is reused in many places on the structure chart.

Fan-Out

- A large number of subordinates associated with a single control should be avoided.
- The general rule of thumb is to limit a control module subordinates to approximately **seven** for low ***fan-out***.
- ***fanin*** is call by other modules.
 - **Create High Fan-In**
- ***fanout*** is call from that module.
 - **Avoid High Fan-Out**

Examples of Fan-in and Fan-out



Assess the Chart for Quality

■ Check list for structure chart quality



- ✓ Library modules have been created whenever possible.
- ✓ The diagram has a high fan-in structure.
- ✓ Control modules have no more than seven subordinates.
- ✓ Each module performs only one function (high cohesion).
- ✓ Modules sparingly share information (loose coupling).
- ✓ Data couples that are passed are actually used by the accepting module.
- ✓ Control couples are passed from “low to high.”
- ✓ Each module has a reasonable amount of code associated with it.

PROGRAM SPECIFICATION

- *Program Specifications* are documents that include explicit instructions on how to program pieces of code.
- There is no formal syntax for program specification.
- Four components are essential for program specification:
 - Program information;
 - Events;
 - Inputs and outputs; and
 - *Pseudocode* – a detailed outline of lines of code that need to be written.
- Additional notes and comments can also be included.

Program Specification 1.1 for ABC System

Module _____
Name: _____
Purpose: _____
Programmer: _____
Date due: _____

C PowerScript HTML/PHP Visual Basic

Events _____

Input Name	Type	Used by	Notes

Output Name	Type	Used by	Notes

Pseudocode _____

Other _____

Module

Name: Find_tune_by_Title

Purpose: Display basic tune information, using a title input by the user

Programmer: John Smith

Date due: April 26, 2009

☐ C

☒ HTML/PHP

☐ Visual Basic

☐ Javascript

Events

search by title push-button is clicked

search by title hyperlink is selected

Input Name:	Type:	Provided by:	Notes:
Tune title	String (50)	Program 1.1.2	

Output Name:	Type:	Used by	Notes:
Tune ID	String (10)	Program 1.1.2	
Not_found	Logical	Program 1.1.2	Used to communicate when tune is not found

Pseudocode

(Find_tune module)

not_found = True

For all tune titles in Available Tunes table

 If user title matches tune title, save tune ID

 not_found = False

 End If

End For

Return

Other

Business rule: If no matching tunes are found, the "Artist of the week" will appear to the user.

Note: A control couple containing a not_found flag should be included from 1.1.2.2 to 1.1.2 to instruct 1.1.2 to display a not found message to the user and the Artist of the week.

(cont'd)

Pseudocode Example

```
Calculate_discount_amount (total_price, discount_amount)
  If total_price < 50 THEN
    discount_amount = 0
  ELSE
    If total_price < 500 THEN
      discount_amount = total_price *.10
    ELSE
      discount_amount = total_price *.20
    END IF
  END IF
END
```

SUMMARY

- **Moving from logical to physical process models.**
 - **Physical DFDs** show implementation details.
- **Structure chart.**
 - The structure chart shows all of the functional components needed in the program at a high level.
- **Building structure chart.**
 - Module, control connection, couples, review.
- **Structure chart design guidelines.**
 - Cohesion, coupling, and fan-in/fan-out.
- **Program specifications.**
 - Program specifications provide more detailed instructions to the programmers.